

Practical 10

```
In [22]: # — Row Transposition Cipher —————  
  
'''  
Logic of Row Transposition Cipher  
  
Key →    3    1    4    2  
        -----  
Row 0 :  H    E    L    L  
Row 1 :  O    W    O    R  
Row 2 :  L    D    x    x  
  
Sort by key (1,2,3,4) → read col 1, col 3, col 0, col 2  
→ EWD + LRx + HOL + LOx = EWDLRXHOLOX  
  
'''  
  
# Step 1: Encrypt  
def encrypt(text, key):  
    text = text.replace(' ', '')  
    cols = len(key)  
    rows = -(-len(text) // cols)          # ceil division  
    text = text.ljust(rows * cols, 'x')    # pad with 'x'  
  
    grid = [list(text[i*cols:(i+1)*cols]) for i in range(rows)]  
    order = sorted(range(cols), key=lambda i: key[i])  
  
    return ''.join(''.join(grid[r][c] for r in range(rows)) for c in order)
```

```
In [23]: # Step 2: Decrypt  
def decrypt(text, key):  
    cols = len(key)  
    rows = len(text) // cols  
    order = sorted(range(cols), key=lambda i: key[i])  
  
    grid = {}  
    idx = 0  
    for c in order:  
        grid[c] = list(text[idx:idx+rows])  
        idx += rows  
  
    return ''.join(grid[c][r] for r in range(rows) for c in range(cols))
```

```
In [24]: # — Demo —————  
msg = "HELLOWORLD"  
key = [3, 1, 4, 2]          # column read order  
  
enc = encrypt(msg, key)  
dec = decrypt(enc, key)
```

```
print("=== Row Transposition Cipher ===")
print(f"Original   : {msg}")
print(f"Key        : {key}")
print(f"Encrypted   : {enc}")
print(f"Decrypted   : {dec}")
```

```
=== Row Transposition Cipher ===
Original   : HELLOWORLD
Key        : [3, 1, 4, 2]
Encrypted  : EWDLRxHOLLOx
Decrypted  : HELLOWORLDxx
```